

DOKUMENTASI PROYEK • TUGAS AKHIR

Sistem Klasifikasi Berita Hoaks

Stacking Ensemble Learning &
Arsitektur Microservices

PENDEKATAN

Stacking Ensemble (NB + LR)

STACK TEKNOLOGI

FastAPI · Docker · Streamlit · MLflow

DEPLOYMENT

Back4App · Hugging Face Hub · DagsHub

1 PENDAHULUAN

Dokumen ini menguraikan **arsitektur sistem**, pemilihan teknologi, dan alur integrasi *end-to-end* untuk pengembangan aplikasi deteksi berita hoaks. Sistem dibangun dengan prinsip **decoupled architecture** — memisahkan beban kerja antara antarmuka pengguna, logika backend, penyimpanan model Machine Learning, dan pelacakan eksperimen (MLOps).

Tujuan Sistem

Membangun pipeline klasifikasi berita hoaks berbahasa Indonesia yang **akurat**, **skalabel**, dan **dapat dipantau secara real-time** menggunakan pendekatan Stacking Ensemble Learning dengan infrastruktur cloud gratis.

2 ARSITEKTUR SISTEM & PEMILIHAN TEKNOLOGI

Untuk memastikan **skalabilitas**, **efisiensi biaya** (infrastruktur gratis), dan **kemudahan pemeliharaan**, proyek ini menggunakan stack teknologi berikut:

Komponen	Teknologi	Fungsi Utama
Model ML	Scikit-Learn	Stacking Ensemble (Naive Bayes + Logistic Regression) & TF-IDF Vectorizer
Penyimpanan Model	Hugging Face Hub	Repositori cloud untuk artefak model.pkl & tfidf_vectorizer.pkl
MLOps & Tracking	DagsHub + MLflow	Log performa model, metrik evaluasi & parameter eksperimen real-time
Backend API	FastAPI + Python 3.10	Menerima request teks, load model dari cloud, kembalikan prediksi JSON
Containerization	Docker	Membungkus lingkungan backend agar konsisten saat di-deploy
Deployment Server	Back4App Containers	Hosting Docker container FastAPI via Public URL
Frontend UI	Streamlit	Antarmuka web interaktif bagi pengguna untuk input teks berita

3 ALUR PENGEMBANGAN SISTEM

Pengembangan sistem dilaksanakan secara bertahap dalam **6 fase** yang berurutan, dari pembuatan model hingga deployment frontend.

FASE 1 Pembuatan & Pelatihan Model (Lokal)

- **Pra-pemrosesan Data** — Teks berita dibersihkan dan dikonversi menjadi representasi numerik menggunakan **TfidfVectorizer**.
- **Pelatihan Stacking Ensemble** — Model dilatih dengan kombinasi *base learners* (Naive Bayes) dan *meta-learner* (Logistic Regression).
- **Penyimpanan Artefak** — Model & vectorizer diekspor ke file .pkl via **joblib** untuk menghindari masalah versi saat unpickling.

FASE 2 Distribusi Model ke Cloud (Hugging Face)

- Repositori model dibuat di **Hugging Face Hub**.
- Artefak model.pkl dan tfidf_vectorizer.pkl diunggah ke repositori.
- Backend dikonfigurasi untuk *pull* file ini secara dinamis via modul huggingface_hub setiap kali API dinyalakan — menjaga repositori aplikasi tetap ringan.

FASE 3 Konfigurasi MLOps (DagsHub & MLflow)

- Repository proyek dihubungkan ke **DagsHub** untuk mengaktifkan MLflow remote tracking server.
- Logika **MLflow** disisipkan di kode backend — setiap klasifikasi mencatat metrik & status eksekusi langsung ke dasbor DagsHub.

FASE 4 Pembangunan Backend API (FastAPI & Docker)

- **Endpoint POST /predict** — Menerima input teks 1D, transform via TF-IDF, lalu masukkan ke model Stacking untuk prediksi.
- **Manajemen Dependensi** — Versi library disamakan ketat di requirements.txt (khususnya scikit-learn==1.7.2) untuk mencegah crash inferensi.
- **Dockerization** — Entry point Dockerfile menggunakan Uvicorn:

```
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "7860"]
```

FASE 5 Deployment Backend (Back4App)

- Repositori GitHub (FastAPI + Dockerfile) dihubungkan ke **Back4App**.
- Layanan *Web Services / Containers* membaca Dockerfile, membangun image, dan mengeksekusinya.
- Backend berstatus "**Available**" dan menghasilkan **Public URL** yang dapat diakses.

FASE 6

Pembangunan Frontend (Streamlit)

- Antarmuka web dibangun menggunakan **Streamlit** (app.py).
- Library requests digunakan untuk mengirim input pengguna sebagai payload JSON ke URL API Back4App.
- Response JSON diproses dan ditampilkan sebagai label **Hoaks / Fakta** di layar.

4 ALUR KERJA APLIKASI (DATA FLOW)

Proses klasifikasi *end-to-end* berjalan dalam urutan berikut:

1	Input User Pengguna mengetik atau menyalin teks berita ke kolom teks di website Streamlit.
2	HTTP Request Streamlit mengirimkan POST request berisi teks tersebut ke endpoint URL Back4App.
3	Inferensi Model FastAPI menerima teks → load model dari Hugging Face → transform via TF-IDF → prediksi kelas berita.
4	Logging MLOps Hasil eksekusi dan metrik dikirim ke DagsHub/MLflow di latar belakang (async).
5	Response ke User FastAPI mengembalikan JSON ke Streamlit, lalu dirender sebagai notifikasi visual Hoaks / Fakta.

5 MANAJEMEN KREDENSIAL (ENVIRONMENT VARIABLES)

Untuk menjaga keamanan, **seluruh token dan API Key** disimpan di dalam pengaturan *Environment Variables* di platform hosting (Back4App / Streamlit Cloud), dan **tidak pernah di-commit ke GitHub**.

Environment Variable	Keterangan
HF_TOKEN	Otentikasi unduh model privat dari Hugging Face Hub.
MLFLOW_TRACKING_URI	Endpoint remote tracking server DagsHub.
MLFLOW_TRACKING_USERNAME	Username akun DagsHub untuk MLflow authentication.
MLFLOW_TRACKING_PASSWORD	Password / token akun DagsHub untuk MLflow.

■ **Penting:** Jangan pernah menyertakan nilai aktual dari variabel-variabel di atas di dalam kode sumber atau file konfigurasi yang di-push ke repository publik. Gunakan mekanisme *secret management* yang disediakan oleh masing-masing platform.

